



Programación Concurrente y Distribuida (CC65)

Trabajo Parcial 2026-1

Secciones: Todas

Profesores: Ms. Carlos Alberto Jara García

Instrucciones

- Desarrollar los casos de uso según indicación de su profesor.
- El trabajo es por grupo de alumnos máximo 3 integrantes
- Se usará software para detección de plagio.

Contexto del Trabajo

La programación concurrente se refiere a la capacidad de un sistema para realizar múltiples tareas de forma coordinada y sincronizadas, lo que es esencial en el contexto actual de desarrollo de software, donde la demanda de sistemas eficientes y escalables es alta. La combinación de programación concurrente con machine learning ofrece varios beneficios en la actualidad.

Estos beneficios se centran en el aprovechamiento eficiente del uso de lenguajes de programación como Go (sin librerías de terceros) con mecanismos como los goroutines para la mejora del rendimiento de las aplicaciones, así como, semáforos para sincronización entre goroutines usando `sync.Mutex`, `sync.WaitGroup` especialmente cuando se trabaja con grandes conjuntos de datos.

Dado que el trabajo con dataset con más de 1MM de registros puede ser altamente costoso, se propone implementar un algoritmo concurrente que use múltiples goroutines, de manera que se optimice el tiempo de ejecución y se mejore la escalabilidad del sistema.

Fuente de datos de referencia, plataforma nacional de datos abiertos de Perú (<https://www.gob.pe/datosabiertos>).

Temas del Trabajo:

Tema: Detección Concurrente de Spam y Anomalías

1. Detector Concurrente de "Red Flags" en el Libro de Reclamaciones (INDECOPI)
 - o Dataset: Registros de quejas y reclamos a nivel nacional (>1M registros).
 - o Aplicación: Implementar un detector concurrente que identifique patrones sospechosos (spam o bots) basados en la repetición de texto y tiempos de envío inusuales para proteger la integridad de las denuncias ciudadanas.
 - o Enfoque: Uso de goroutines para procesar lotes de texto en paralelo y `sync.Mutex` para actualizar el contador de alertas globales.
2. Auditoría Concurrente de Comentarios en Consultas Públicas (PCM)
 - o Dataset: Comentarios y participaciones en la Plataforma Digital Única.
 - o Aplicación: Filtrado masivo y concurrente de reseñas de servicios públicos para detectar "astroturfing" (bots simulando apoyo popular), analizando la frecuencia de IPs o usuarios en ventanas de tiempo críticas.
 - o Enfoque: Implementación de un Pipeline en Go para limpieza, filtrado y detección de duplicidad

Tema: Preprocesamiento Concurrente de Texto (NLP)

1. Procesador Masivo del Diario Oficial El Peruano para Minería de Datos Legal
 - Dataset: Resoluciones, decretos y normas legales digitalizadas (>1M documentos/entradas).
 - Aplicación: Construir un pipeline concurrente de limpieza, tokenización y lematización a gran escala para preparar el dataset para futuros modelos de IA gubernamental.
 - Enfoque: Comparar el rendimiento secuencial vs. concurrente mediante el cálculo de Speedup tras procesar millones de tokens
2. Limpieza Paralela de Expedientes Judiciales para Búsqueda Semántica
 - Dataset: Metadatos y resúmenes de sentencias del Poder Judicial.
 - Aplicación: Preprocesamiento concurrente de millones de registros judiciales para alimentar sistemas de búsqueda inteligente, asegurando la anonimización de datos sensibles en paralelo.
 - Enfoque: Simulación en Promela de la gestión de memoria y buffers de texto para evitar condiciones de carrera

Tema: Recomendadores Híbridos Concurrentes

1. Matcher Concurrente para Contrataciones del Estado (SEACE)
 - Dataset: Órdenes de compra y perfiles de proveedores (>1M registros).
 - Aplicación: Sistema híbrido que recomiende oportunidades de negocio a PYMES combinando el contenido de la convocatoria con el historial de ventas (colaborativo) del proveedor, ejecutando ambos enfoques en goroutines independientes
 - Enfoque: Uso de sync.WaitGroup para sincronizar la fusión de resultados de los dos modelos antes de mostrar la recomendación final
2. Recomendador Híbrido de Becas y Créditos Educativos (PRONABEC)
 - Dataset: Postulaciones históricas y requisitos de programas de becas.
 - Aplicación: Desarrollo de un motor concurrente que combine el perfil socioeconómico del estudiante con la descripción de las becas para sugerir la mejor opción de financiamiento educativo.
 - Enfoque: Verificación con Spin de que el acceso concurrente a la base de datos de perfiles no genere bloqueos (deadlocks)

Entregables

El trabajo está dividido en 3 entregables:

- Entregable 1 – PC1 (Semana 3): Investigación bibliográfica (3 papers top ranking), selección y limpieza del dataset (>1M registros) y modelado inicial en Promela para simular la lógica de sincronización.
- Entregable 2 – PC2 (Semana 5): Implementación en Go (secuencial vs. concurrente) usando patrones como Worker Pools o Pipelines. Cálculo de Speedup (T-Secuencial / T-Concurrente) y media recortada tras múltiples ejecuciones.
- Entregable 3 – TP (Semana 7): Verificación formal en Spin (ausencia de deadlocks y exclusión mutua), informe de IA con el detalle técnico de la solución, análisis de cada punto indicado en el informe, con las conclusiones y recomendaciones del alumno.

Rúbrica de calificación

- Entregable 1:
 - (10 puntos) Calidad de la documentación bibliográfica revisada (Papers y artículos científicos de buena reputación, análisis y conclusiones del alumno)
 - (2 puntos) Análisis de 1 caso de uso y sustentar su elección y explicación del dataset
 - (3 puntos) Explicación del procedimiento de limpieza del dataset
 - (5 puntos) Modelado inicial en Promela para simular la lógica de sincronización
- Entregable 2:
 - (4 puntos) Implementación de código secuencial y concurrente de los algoritmos.
 - (4 puntos) Explicación del algoritmo y el uso de los mecanismos de sincronización utilizado, incluyendo el uso de patrones como Worker Pools o Pipelines.
 - (4 puntos) Cálculo de Speedup ($T\text{-Secuencial} / T\text{-Concurrente}$) y media recortada tras múltiples ejecuciones.
 - (5 puntos) Análisis de speedup, scalability y trade-offs entre modelos concurrentes y secuenciales
 - (3 puntos) Análisis de uso y rendimiento de los recursos de cómputo utilizado.
- Entregable 3:
 - (5 puntos) Verificación formal en Spin (ausencia de deadlocks y exclusión mutua) explicación de los resultados.
 - (5 puntos) Trabajo con la IA para analizar el código subido a GitHub y elaborar un informe en formato .md (Markdown) con el detalle técnico que determine los GAPs existentes, tanto en calidad de código, seguridad, patrones de concurrencia u otro aspecto relevante.
 - (5 puntos) Análisis con las conclusiones y recomendaciones del alumno.
 - (5 puntos) Presentación de informe final y sus anexos.

Documentación

1.- Presentar un informe entregable PC1 – **Fecha de entrega semana 3:**

- a. Carátula
- b. Índice
- c. Resumen del trabajo
- d. Objetivos del trabajo
- e. Explicación de cada tema investigado (3 temas por grupo)
Por cada paper deberá presentar una tabla con la siguiente información:
 - Título del paper
 - Autores
 - Año de publicación
 - Motivación
 - Problema que resuelve
 - Propuesta de solución
 - Algoritmos usados, pseudocódigo
 - Descripción de servicios, framework, bibliotecas usadas, código, lenguaje de programación, patrones.

- Opinión crítica de su parte (estudiante) de los beneficios o perjuicios del uso de la programación concurrente.
 - Link de acceso al paper.
- f. Análisis de 1 caso de uso y sustentar su elección y explicación del dataset
 - g. Explicación del procedimiento de limpieza del dataset
 - h. Modelado inicial en Promela para simular la lógica de sincronización

2.- Presentar un informe entregable PC2 – **Fecha de entrega semana 5:**

- i. <Informe inicia con los puntos de la PC1 y correcciones>
- j. Implementación de la solución en Go (secuencial vs. concurrente)
- k. Explicación del algoritmo y el uso de los mecanismos de sincronización utilizado, incluyendo el uso de patrones como Worker Pools o Pipelines.
- l. Cálculo de Speedup (T-Secuencial / T-Concurrente) y media recortada tras múltiples ejecuciones.
- m. Análisis de speedup, scalability y trade-offs entre modelos concurrentes y secuenciales
- n. Análisis de uso y rendimiento de los recursos de cómputo utilizado.

2.- Presentar un informe entregable TP – **Fecha de entrega semana 7:**

- i. <Informe inicia con los puntos de la PC1 + PC2 y sus correcciones>
- j. Verificación formal en Spin (ausencia de deadlocks y exclusión mutua)
- k. Elaborar un prompt estructurado, seleccione un modelo de IA y analice el código subido a GitHub y elaborar un informe en formato .md (Markdown) con el detalle técnico que determine los GAPs existentes tanto en calidad de código, seguridad, patrones de concurrencia u otro aspecto relevante.
- l. Análisis con las conclusiones y recomendaciones del alumno.
- m. Referencias bibliográficas (APA)
- n. Subir su código fuente al GitHub (acceso público) y evidenciar los commits en la rama de desarrollo. **Se bajará 10 puntos menos si no se presenta esta evidencia o de identifica que fue editado luego de la fecha de entrega.**
- o.** Enlace de video de máximo 5 minutos demostrando cada integrante conocimiento del tema y presentando los resultados del trabajo, discusión crítica sobre limitaciones, escalabilidad y posibles mejoras. (publicarlo en un repositorio en la nube para acceder y visualizar a través de la url que deberá colocarlo como anexo en el informe del trabajo). **Se bajará 5 puntos menos si no se presenta el video.**
- p.** Hoja de participación del grupo.

Presentación

Cada alumno sin excepción deberá subir al aula virtual su informe en formato Word. El nombre del archivo deberá tener el formato según el entregable:

- CC65-PC1-202601-[Código de alumno]
- CC65-PC2-202601-[Código de alumno]
- CC65-TP-202601-[Código de alumno]

El alumno que no suba su informe al aula virtual en el plazo indicado tendrá como nota = 0

Tome sus precauciones para evitar cualquier inconveniente.